

Computing Methods for Physics 1 – 17 January 2024

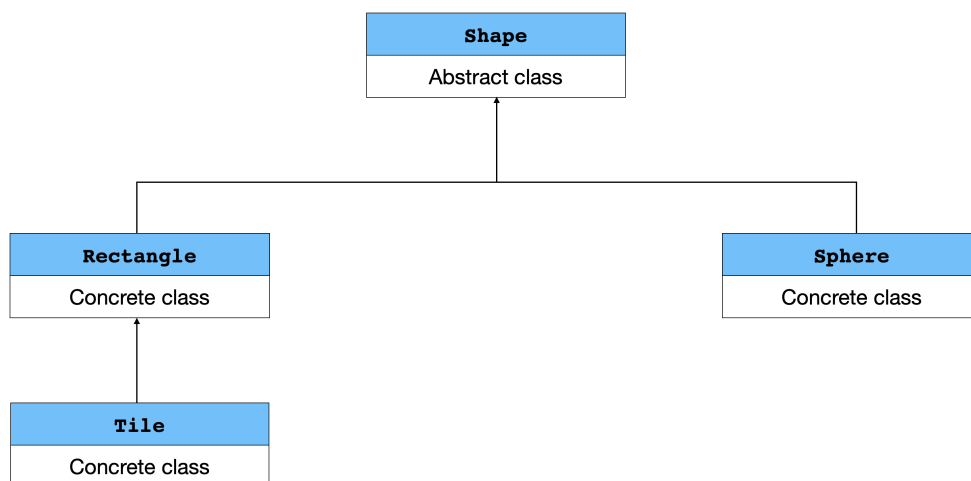
Your exam material (code files, plots, datafiles, etc.) must be submitted via google classroom by 14:15 as a single zip file.

C++ evaluation will be based on: correct syntax, proper return types, proper arguments of functions, data members and class interfaces, setters/getters, unnecessary void functions, correct mathematical expressions, comments throughout the code, separation of class implementations and interfaces.

Python evaluation will be based on: correct syntax, avoiding C-style loops, using Python features in general, comments throughout the notebook/scripts, labels, legends and plot styling and clarity in general.

Part 1 – C++

Implement the following classes and inheritance hierarchy.



- **Shape** must have the string attribute **label** and it must require any concrete class inheriting from it to specify a **print** method.
- Design the concrete class **Rectangle** to represent rectangles; this includes calculating and storing their area and perimeter.
- Design the concrete class **Sphere** to represent spheres; this includes calculating and storing their surface area and volume.
- The concrete class **Tile** is intended to represent physical objects. It must have the attribute **mass** and a method to determine the average density. It must also have the class method **dilate** to vary all attributes of a physical object when its temperature changes by ΔT [assume that objects are isotropic and recall linear dilation, i.e., $\ell = \ell_0(1 + \lambda\Delta T)$, where λ is the linear dilation coefficient].

Part 2 – Python

A compact binary is a system of two black holes and/or neutron stars bound gravitationally and therefore orbiting each other. Ignoring the individual rotation angular momenta, when a compact binary forms it is characterized by the two masses m_1 and m_2 , the semi-major axis of the orbit $a(0)$, and the eccentricity of the orbit $e(0)$. Gravitational-wave emission drives both a and e to 0 over time. The equations that govern the orbital evolution are

$$\frac{da}{dt} = -\frac{\beta}{a^3}\Psi(e) \quad \text{with} \quad \Psi(e) = \frac{1 + 73e^2/24 + 37e^4/96}{(1 - e^2)^{7/2}} \quad (1)$$

$$\frac{de}{dt} = -\frac{19}{12}\frac{\beta}{a^4}\Phi(e) \quad \text{with} \quad \Phi(e) = \frac{(1 + 121e^2/304)e}{(1 - e^2)^{5/2}} \quad (2)$$

where

$$\beta = \frac{64}{5} \frac{G^3 \mu M^2}{c^5} \quad (3)$$

$$M = m_1 + m_2 \quad (4)$$

$$\mu = \frac{m_1 m_2}{M}. \quad (5)$$

- Ask the user to provide m_1 , m_2 , $a(0)$, and $e(0)$ and then integrate the differential equations above. [For reference, neutron star masses are between 1 and 2.5 solar masses ($M_\odot \simeq 2 \cdot 10^{30}$ kg); for black holes, consider masses between $5M_\odot$ and $100M_\odot$.]
- Inform the user about how much time elapses until a crosses a specific value $\bar{a}$, also provided by the user.
- Post-process $a(t)$ to determine the time evolution of the gravitational-wave frequency

$$f_{\text{GW}} = \frac{1}{\pi} \sqrt{\frac{GM}{a^3}}. \quad (6)$$

- Store $a(t)$, $e(t)$, and $f_{\text{GW}}(t)$ to a single binary file.
- Plot $a(t)$, $e(t)$, and $f_{\text{GW}}(t)$.
- Apply your code to a $1.4M_\odot + 1.4M_\odot$ system with $e(0) = 0.01$ and $f_{\text{GW}}(0) = 30$ Hz.